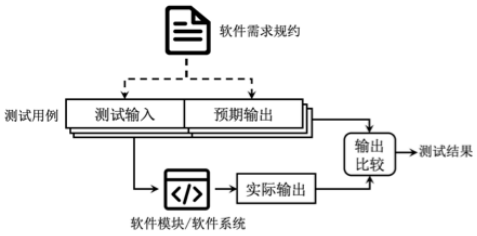
Software Testing

- We got a software
- We throw some sample data at it
- We check whether the software performed as we expected



Early Testing Practice @ Google

- In Google's early days, engineer-driven testing was often assumed to be of little importance.
-
- Teams regularly relied on smart people to get the software right.
- A few systems ran large integration tests, but mostly it was the Wild West.

One product that suffers the most is **Google Web Server (GWS)**, which is the web server responsible for serving Google Search queries

The Story of Google Web Server (GWS)

- Back in 2005, as the project swelled in size and complexity, productivity had slowed dramatically.
- Releases were becoming buggier, and it was taking longer and longer to push them out.
- Team members had little confidence when making changes to the service, and often found out something was wrong only when features stopped working in production.

At one point, more than 80% of production pushes contained user-affecting bugs that had to be rolled back.

To address these problems, the tech lead of GWS decided to institute a policy of

engineer/developer-driven, automated testing

开发者驱动、自动测试

Developer Driven Testing

Every coded feature should be tested completely once, by the developer responsible for implementing it

- In Developer Driven Testing (DDT), developers write their tests by themselves, only when they feel they're done with the code.
- They are the people who have to verify that the code is "good enough," not the testers.
- The purpose of this approach is to make developers responsible for their code.

At Google, all new code changes were required to include tests

Automated Testing

Manual testing won't scale for modern software development

- Thousands of lines of code
- Hundreds of libraries and frameworks
- Multiple platforms & languages
- Uncountable number of configurations
- Frequent updates or new versions
- Delivered via unreliable networks

At Google, all new code changes were required to include tests, which run automatically and continuously (e.g., thousands of times per day)

Outcome

- Within a year of instituting this policy, the number of emergency pushes *dropped by half*.
- This drop occurred despite the fact that the project was seeing a record number of new changes every quarter.
- Even in the face of unprecedented growth and change, testing brought renewed productivity and confidence to one of the most critical projects at Google.

Today, GWS has tens of thousands of tests, and releases almost every day with relatively few customer-visible failures.

Test Case vs. Test Suite

- **Test case** (测试用例): a sequence of actions necessary to verify a specific functionality or feature of the software.
- A test case specifies the prerequisites, post conditions, steps, and data required for feature verification.

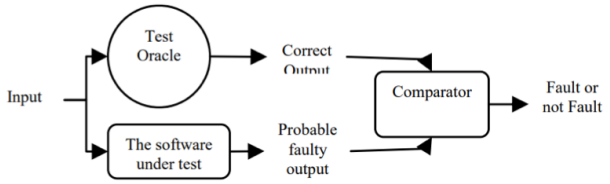
Test case example: Check results when a valid Login Id and Password are entered.

- **Test suites** (测试套件、测试集) are the logical grouping or collection of test cases to run a single job with different test scenarios.
- Example: a test suite for product purchase has multiple test cases

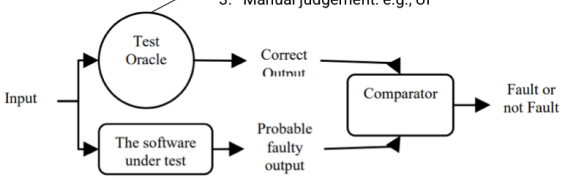
A "product purchase" test suite
- Test case 1: Login
- Test case 2: Adding products
- Test case 3: Payment
- Test case 4: Logout

Test Input vs. Test Oracle

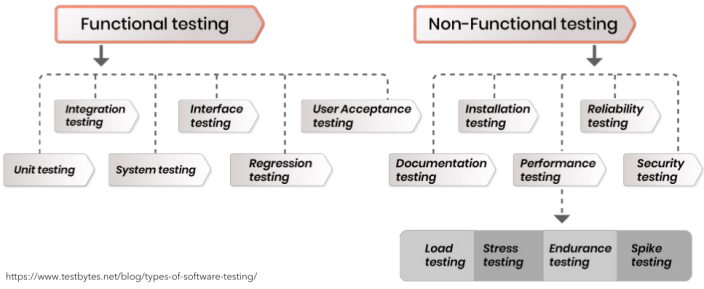
Test Oracle (测试预言、测试判断准则) is a fault free source of expected outputs: it accepts every test input specified in software's specification and should always generate a correct result



1. Expected output: e.g., calculator
2. Business rules: e.g., items in a shopping cart cannot exceed 100
3. Manual judgement: e.g., UI



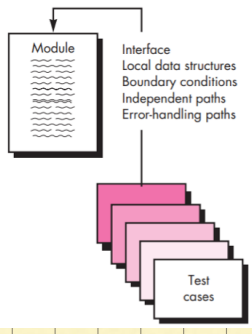
Types of Software Testing



Unit Testing (UT)

UNIT TESTING

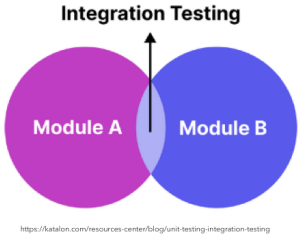
- Unit testing focuses verification effort on the smallest unit of software design, e.g., methods, classes
- Unit tests focus on the internal processing logic and data structures within the boundaries of a component.
- By isolating each unit and testing it independently, unit testing can be conducted in parallel for multiple components.



Integration Testing (IT)

INTEGRATION TESTING

- Unit testing ensures that components work individually
- Integration testing verifies the interactions and behavior of multiple components/modules working together (though interfaces)



https://katalon.com/resources-center/blog/unit-testing-integration-testing

Test Case ID	Test Case Objective	Test Case Description	Expected Result
1	Check the interface link between the Login and Mailbox module	Enter login credentials and click on the Login button	To be directed to the Mail Box
2	Check the interface link between the Mailbox and Delete Mails Module	From Mailbox select the email and click a delete button	Selected email should appear in the Deleted/Trash folder

An example of integrating testing involving a website that features "Log-in Page," "Mailbox," and "Delete E-mails" functions.

System Testing

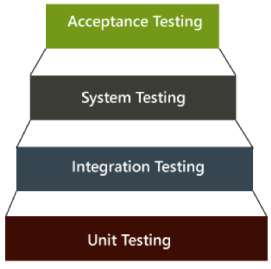
- System testing aims at testing that the product fulfills the business specifications and is ready for deployment
- System testing is generally executed after the integration test and before the delivery of the software product
- System testing is often conducted in an environment closely resemble the production environment
- System testing is usually performed by a dedicated testing team
- System testing may include functional testing, performance testing, security testing, etc.



Acceptance Testing

ACCEPTANCE TESTING

- The last phase of software testing performed after System Testing and before making the system available for actual use.
- Acceptance testing is used to determine whether the product is working for the end-users correctly
- Acceptance testing may involve end users perform realistic tasks using real data



Test Size

How much resources are consumed by a test

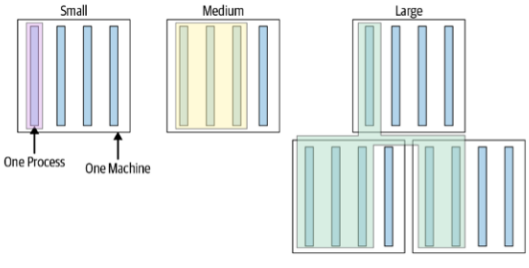
Small tests
- run in a single thread/process, no blocking calls (e.g., no network calls)
- provide a safe "sandbox"
- fast, effective, and reliable

How much resources are consumed by a test

Medium tests
- Run on a single machine
- can span multiple processes, use threads, and can make blocking calls (e.g., network calls only to localhost, but not to remote machines via network)
- Enable testing the integration of multiple components (database, UI, server, etc.)

How much resources are consumed by a test

Large tests
- Run on a multiple machines over network
- More about validating configurations instead of piece of code
- Reserved for full system end-to-end tests

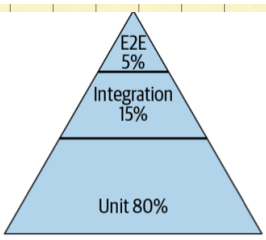


Test Scope

TEST SCOPE How much code a test intends to validate

- Narrow-scoped tests
  - E.g., unit tests
  - Validate the logic in a small, focused part of the codebase (e.g., class, method)
- Medium-scoped tests
  - E.g., integration tests
  - Verify interactions between a small number of components (e.g., server + database)
- Large-scoped tests
  - E.g., end-to-end tests or system tests
  - Validate several parts of the system or the entire system

- 80% of tests being narrow-scoped unit tests that validate the majority of business logic
- 15% medium-scoped integration tests that validate the interactions between two or more components
- 5% end-to-end tests that validate the entire system.



Google's test pyramid

Black-Box Testing vs. White-Box Testing





Black-box testing
- Based on software specification
- Internal program structure is unknown
- Cover as much specified behaviors as possible

Black-box testing
- Pros: simplicity, realistic results (simulate end users)
- Cons: limited coverage, incomplete testing

White-box testing
- Based on software code
- Internal program structure is known
- Cover as much coded behaviors as possible

White-box testing
- Pros: comprehensive testing, early defect detection
- Cons: requires technical expertise, expensive, limited real-world simulation.

Example

Specification: inputs an integer and prints it



Test possible integers based on specification, e.g., 1, 0, -1

Miss the bug

Specification: inputs an integer and prints it

Possible implementation

```
void print(num){
        if(param<1024) printf("%d", num);
        else printf("%d KB", num/124)
}
```


Test every possible code path, e.g., 1000, 2000

Detect the bug

Possible implementation

```
int func(int num){
        int result;
        result = num/2;
        return result;
}
```



Test every possible code path, e.g., 2, 200

Miss the bug

Specification: inputs an integer and returns half of its value for even integers, and the same value otherwise

Possible implementation

```
int func(int num){
        int result;
        result = num/2;
        return result;
}
```



Test every behavior in the specification, e.g., 2, 3

Detect the bug

White-Box Testing

- One of the main goals of white box testing is to cover the source code as comprehensively as possible.
- Assumption: executing the faulty statement is a necessary condition for revealing a fault
- Code coverage is a metric that shows how much of an application's code has tests checking its functionality

– Statement Coverage

- Statement coverage aims at executing all possible statements of the source code at least once.
- It is used to calculate the total number of executed statements in the source code out of total statements present in the source code.

$$\text{Statement coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} * 100$$

- Branch Coverage

- A "branch" is one of the possible execution paths the code can take after a decision statement—e.g., an `if` statement—gets evaluated.
- Branch coverage is a requirement that, for each branch in the program, each branch have been executed at least once during testing (i.e., each branch condition must have been TRUE at least once and FALSE at least once)

```
Branch coverage = ( # of executed branches / # of total branches )  x 100%
```

- Condition Coverage

- Condition coverage is also known as Predicate Coverage, in which each one of the boolean expression (condition) should have been evaluated to both TRUE and FALSE.

```
Condition coverage = ( # of conditions that are both T and F / # total conditions )  x 100%
```

White-Box Testing

Types of coverage

- Statement coverage
- Branch coverage
- Condition coverage
- B&C coverage
- Path coverage
- Multiple condition coverage
- Finite state machine coverage
- ……

Black-Box Testing

- Test Data Selection

Exhaustive Testing

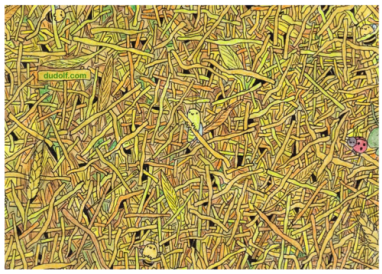
- Test every possible input
- Not feasible w.r.t. time

Example: add(int a, int b)

- $2^{32} * 2^{32} = 2^{64}$ or 10^{19} tests
- Assume 1 test / nanosecond ~ 10^9 tests / second
- 10^{10} seconds for exhaustive testing

600 years

Random Testing: Pick inputs randomly

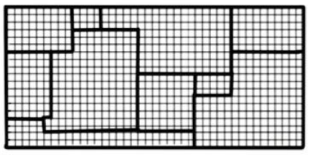


Pick a test that reveals the bug is like finding a needle in a haystack

随机不一定能随到痛点.

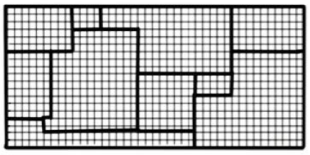
Partition Testing:

- This method divides the input data of software into different equivalence data classes (等价类).
- In general, input could be partitioned into valid/invalid equivalence class
- Test inputs can be selected from each partition to reduce time required for testing



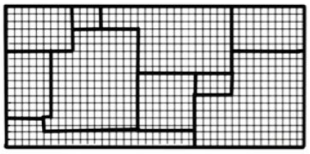
Equivalence Partition Hypothesis:

- If one condition/value in a partition passes all others will also pass.
- Likewise, if one condition in a partition fails, all other conditions in that partition will fail.



Boundary Values:

- Errors tend to occur at boundary of a (sub)domain
- Boundary testing is the process of testing between extreme ends or boundaries between partitions of the input values, which increases the chance of revealing faults



Example

EXAMPLE

Order Pizza: 1 Submit

Specification

- Pizza values 1 to 10 is considered valid. A success message is shown.
- Pizza value 11 to 99 are considered invalid for order and an error message will appear, "Only 10 Pizza can be ordered"

Invalid	Valid	Invalid	Invalid
0	1 10	11 99	100
Partition 1	Partition 2	Partition 3	Partition 4

Unchanging Tests

- Ideally, after a test is written, it never needs to change unless the requirements of the system under test change
- Maintainable tests "just work": after writing them, engineers don't need to think about them again until they fail, and those failures indicate real bugs with clear causes.

After writing a test, you shouldn't need to touch that test again as you refactor the system, fix bugs, or add new features.

Refactorings: refactorings don't change the systems' behaviors, existing tests should remain unaffected
New features: new features may require new tests, but existing tests should remain unaffected
Bug fixes: like new features, may require new tests, but existing tests should remain unaffected

Only changing the system's existing behavior would require the updates to existing tests

仅当系统行为改变时才更新测试

Test via Public APIs

- Public APIs are exposed to users
- Public APIs change much less frequently than internal implementations
- Testing public APIs mimic how users use the system

TEST VIA PUBLIC APIS

```
@Test
public void emptyAccountShouldNotBeValid() {
  assertThat(processor.isValid(newTransaction().setSender(EMPTY_ACCOUNT)))
      .isFalse();
}
```

X

Remove "private" modifier and test the internal implementation logic directly.

Example 12-1. A transaction API

```
public void processTransaction(Transaction transaction) {
  if (isValid(transaction)) {
    saveToDatabase(transaction);
  }
}

private boolean isValid(Transaction t) {
  return t.getAmount() < t.getSender().getBalance();
}

private void saveToDatabase(Transaction t) {
  String s = t.getSender() + "," + t.getRecipient() + "," + t.getAmount();
  database.put(t.getId(), s);
}

public void setAccountBalance(String accountName, int balance) {
  // Write the balance to the database directly
}

public void getAccountBalance(String accountName) {
  // Read transactions from the database to determine the account balance
}
```

```
@Test
public void shouldTransferFunds() {
  processor.setAccountBalance("me", 150);
  processor.setAccountBalance("you", 20);

  processor.processTransaction(newTransaction()
      .setSender("me")
      .setRecipient("you")
      .setAmount(100));

  assertThat(processor.getAccountBalance("me")).isEqualTo(50);
  assertThat(processor.getAccountBalance("you")).isEqualTo(120);
}
```

√

The test access the system in the same manner as the users would via public APIs

Test Behaviours, Not Methods

- Common patten: every method has a corresponding @test method
- Convenient at first
- But as the method grow more complex, the test also grows in complexity and becomes less maintainable

```
public void displayTransactionResults(User user, Transaction transaction) {
  ui.showMessage("You bought a " + transaction.getItemName());
  if (user.getBalance() < LOW_BALANCE_THRESHOLD) {
    ui.showMessage("Warning: your balance is low!");
  }
}
```

The method to be tested

```
@Test
public void testDisplayTransactionResults() {
  transactionProcessor.displayTransactionResults(
      newUserWithBalance(
          LOW_BALANCE_THRESHOLD.plus(dollars(2))),
      new Transaction("Some Item", dollars(3)));

  assertThat(ui.getText()).contains("You bought a Some Item");
  assertThat(ui.getText()).contains("your balance is low");
}
```

X

A method-driven test: as the method under test becomes more complex and implements more functionality, its unit test will become increasingly convoluted and grow more and more difficult to work with.

```
@Test
public void displayTransactionResults_showsItemName() {
  transactionProcessor.displayTransactionResults(
      new User(), new Transaction("Some Item"));
  assertThat(ui.getText()).contains("You bought a Some Item");
}
```

```
@Test
public void displayTransactionResults_showsLowBalanceWarning() {
  transactionProcessor.displayTransactionResults(
      newUserWithBalance(
          LOW_BALANCE_THRESHOLD.plus(dollars(2))),
      new Transaction("Some Item", dollars(3)));
  assertThat(ui.getText()).contains("your balance is low");
}
```

√

A behavior-driven test: rather than writing a test for each method, write a test for each behavior.